

IRE Assignment 2 — Design Note

Chaitanya Shah · done individually · github.com/ChaitanyaShah21/A2

1. What I built, and why

1.1 The problem and the two-stage design

In Assignment 1, we used BM25 and semantic embeddings to find out the top articles most likely to be clicked by a user. However, those only consider things like words used in the body or title, or how relevant the article is to what the user already likes. Those are good, but not the whole picture. Features like freshness and popularity also have a major hand in whether an article is clicked or not: 92.7% of MIND and 93.5% of EB-NeRD clicks were on fresh articles. However, calculating these new features for all the articles is very expensive. Therefore, we use a two-stage process. First, we retrieve the top 100 articles using A1's BM25 or semantic embeddings. Next, we run the re-ranker with the new features on only those 100 articles. For the datasets, I chose MIND (65K articles and 230K impressions) and EB-NeRD (125K articles and 1.2M impressions, which is a 5% sample of users from the EB-NeRD large dataset). The full version wouldn't fit in memory on the device, and this sample was enough for a confidence interval of about ± 0.001 on AUC. Taking a larger sample would have added to the cost of later steps with very little gain.

1.2 Features and the allowlist

Feature	What it measures	MIND	EB-NeRD
bm25	A1 lexical (word-overlap) score between user history and candidate	✓	✓
cos_inf	A1 semantic score: similarity of candidate to the user's whole click history	✓	✓
cos_short, cos_medium	Same similarity, but recent clicks weigh more (short / medium half-life)	✓	✓
cat_share_short / medium / inf	Share of the user's clicks in the candidate's category, recency-weighted	✓	✓
freshness_hours	Hours since the earliest evidence the article existed	✓	✓
exposure_share_1h, _24h	Share of all impressions in the last 1 h / 24 h that showed this article	✓	✓
hours_since_last_click	Time since the user last clicked anything	—	✓
session_position, minutes_since_session_start, device_type	Where the user is inside the current visit	—	✓
20 rack columns (Section 2.2)	Each feature re-expressed relative to the other candidates in the same list	✓	✓

Table 1. Re-ranker features: one row per (impression, candidate article). 10 common + 4 EB-NeRD-only, plus 20 rack columns.

Assignment 1 had two relevance scores: bm25 and cos_inf. We build new features on top of these ones, so the improvements or changes made are directly attributed to the new features added. The new features tell us what the original two could not: how fresh the article is (freshness_hours), to how many people it is being shown right now (exposure_share), whether the user is as interested in the topic as they were before (cos_short / cos_medium decay versions), and where they are in their visit (session features, EB-NeRD only).

Popularity is taken as a share rather than a raw count to normalize for the fact that the number of

impressions differs between training and leaderboard week data. Training on raw counts would make the model associate a particular number with specific popularity values, which breaks when the load on leaderboard data is different from what it was during training. Ratios remain consistent irrespective of traffic load.

We use an allowlist instead of a denylist to ensure that any feature that leaks future information is not accidentally included for training. For example, the `total_pageviews` column in the `articles` file counts views across the whole dataset, including days after the prediction time, which would allow the model to see into the future. These features are not used in training and are only used for the Q9 tests.

1.3 The re-ranker model

For the re-ranker, I used a gradient-boosted decision tree model using LightGBM with LambdaRank as the training objective. We chose LambdaRank over standard binary click/no-click classification because predicting whether an individual article gets clicked in isolation doesn't match the actual task: what matters is ordering the list so that the clicked article is ranked above the unclicked candidates on that same shelf. The model was trained on the train split, early stopping on the validation split decided when to stop adding trees, and final results were measured on a completely separate test split so the evaluation wasn't optimistic. On MIND, the plain model stopped learning after only 18 trees because it quickly ran out of useful signal, but once rack features were added, it continued improving all the way to 209 trees. Note that while both datasets use recency, MIND lacks timestamps so recency only reflects list position, whereas EB-NeRD uses actual timestamps.

2. Baseline vs improved: ablation and confidence intervals

2.1 Q3: reproduce the neural baseline (NRMS), add one principled change

Comparison (paired Δ AUC, test split)	Δ AUC [95% CI]	CI excludes 0?
EB-NeRD: +freshness +exposure vs NRMS	+0.0864 [0.0852, 0.0877]	yes
EB-NeRD: +freshness only vs NRMS	+0.1141	yes
MIND, sentence-level encoder, 3 epochs	-0.0002 [-0.0011, 0.0008]	no
MIND, sentence-level encoder, 6 epochs	-0.0001 [-0.0011, 0.0009]	no
MIND, word-level encoder (as in the paper)	+0.0032 [0.0018, 0.0045]	yes

Table 2. Adding the time term to NRMS. Encoder check: word-level baseline 0.6129 vs sentence-level 0.6146 (3 ep) / 0.6180 (6 ep) — the substitution cost no accuracy.

NRMS is a neural baseline that encodes news titles using multi-head self-attention to capture user interest from clicked words, but like BM25, it has no awareness of time. Based directly on our Assignment 1 finding that over 92% of clicks go to fresh articles, I made one principled change: feeding article freshness and exposure share directly into the model as extra inputs. On EB-NeRD, this gave a massive, statistically significant boost (+0.0864 Δ AUC [0.0852, 0.0877]), while on MIND the gain was much smaller (+0.0032 with the word-level encoder), possibly because MIND's candidate lists are already largely composed of fresh articles. We also encountered a clear validation-vs-test trap: the word-level encoder has 11M parameters and looked +0.0198 better on the validation split, but it failed to generalize and showed no advantage on the test split. This proved that evaluating on the split used to make design choices gives an overly optimistic picture, and on test, the word-level model was not better.

2.2 The re-ranker, before and after rack normalisation

Metric	MIND Δ [95% CI]	EB-NeRD Δ [95% CI]
AUC	+0.0223 [0.0195, 0.0250]	+0.0202 [0.0194, 0.0210]

MRR	+0.0220 [0.0193, 0.0247]	+0.0234 [0.0222, 0.0245]
nDCG@5	+0.0240 [0.0215, 0.0265]	+0.0225 [0.0215, 0.0235]
nDCG@10	+0.0204 [0.0182, 0.0226]	+0.0210 [0.0202, 0.0219]

Table 3. Rack features vs the shipped re-ranker, paired bootstrap, 1,000 resamples, test split. Every interval excludes zero. Absolute AUC: MIND 0.6144 → 0.6367, EB-NeRD 0.7428 → 0.7629.

MIND slice (AUC change vs A1)	before rack	after rack
head-exposure impressions (most-shown articles)	-0.0127	+0.0162
cold users (little or no history)	-0.0113	+0.0185

Table 4. The two slices where the first A2 model was worse than A1 became the two largest gains.

The shelf problem happens because a user only chooses among the candidate articles shown together on that specific impression 'shelf', but raw feature values vary heavily across different shelves. A user with a long history gets high BM25 scores across all candidates while a new user gets low ones, so a raw score like 12 doesn't tell the model if an article is the top match on that shelf or the bottom one (73.8% of BM25 variation was between shelves). To fix this, I added rack normalization features that express each score relative to its shelf using percentile rank and z-score, so relative ranking is consistent everywhere. This produced clear gains across all metrics on both datasets (MIND AUC +0.0223, EB-NeRD AUC +0.0202), and it repaired the exact two slices where A2 had been losing to A1—head-exposure articles and cold users—turning them into our largest improvements. Using paired bootstrap evaluation over 1,000 resamples allowed impression-level difficulty to cancel out, yielding tight confidence intervals that clearly exclude zero.

2.3 External check: the leaderboards

Dataset	Before (20 Sep)	After rack (26 Sep)	Leaderboard Δ	Offline test
MIND	0.6181, rank 69	0.6319, rank 63	+0.0138	0.6367 (Δ +0.0223)
EB-NeRD	0.7397, rank 77	0.7621, rank 74	+0.0224	0.7629 (Δ +0.0202)

Table 5. Codabench AUC. EB-NeRD was scored by a self-hosted worker using the organisers' published queue setup, because the competition's own workers are retired. Screenshots: [figures/mind_rack.png](#), [ebnerd_rack.png](#).

Submitting our rack-normalized models to Codabench gave an external check on whether offline gains hold up on unseen test sets. On EB-NeRD, our offline evaluation was completely confirmed: the offline test score of 0.7629 predicted the online leaderboard score of 0.7621 to within 0.0008, showing our evaluation setup is trustworthy. On MIND, the leaderboard confirmed a real gain (+0.0138 AUC, improving our rank from 69 to 63), but it only captured about 62% of the improvement predicted by offline testing (+0.0223). This difference is likely because our offline MIND test split covers only the last 12 hours of a single day (21,947 impressions), which cannot account for the broader week-long distribution shifts seen across the 2.37 million impressions on the leaderboard.

3. Serving and scale

Measure (MIND unless stated)	Before	After
Feature stage p50 latency	252.16 ms	3.18 ms
Total p99 latency	509.67 ms	30.53 ms
EB-NeRD p50 speed-up	—	27.6–33.9x
Cost per 1,000 queries	\$0.003078	\$0.000162
Index memory footprint	MIND 283 MB	EB-NeRD 476 MB

Table 6. Serving measurements.

p99 latency, ms	warm (memory loaded)	cold (fresh start)
stage 1: retrieve (not changed by this work)	24.38	121.49
online features (what was changed)	6.70	6.76
online total	48.93	125.47

Table 7. Warm vs cold. The feature stage is fast in both; the cold total fails a 100 ms target because of stage 1.

The initial serving pipeline was too slow for production—p50 feature latency was 252.16 ms and total p99 latency was 509.67 ms—because it naively reused batch Polars code, executing 66 separate table queries on every single request. I fixed this by precomputing user profiles in a nightly offline batch (50K MIND users in 18.7 seconds, or 0.37 ms each), meaning the online server only needs to do fast dictionary lookups. The cost of this fix is that user profiles can be up to 24 hours stale, which is a real trade-off since click patterns on news are time-sensitive. Looking at latency honestly, our optimizations solved the feature stage bottleneck (dropping it to ~6.7 ms p99), but the system only meets the 100 ms p99 target when warm (48.93 ms total); on cold starts, it fails at 125.47 ms because stage 1 retrieval takes 121.49 ms loading from disk. This means production instances must be pre-warmed before taking user traffic.

4. Where it breaks at 10x

Component	At 10x	Kind of limit
Exposure index (who was shown what, when)	~2 billion entries = 16 GB, more than the 11 GB RAM — breaks first	memory (provisioning)
Nightly user profiles	~8M users x 3.4 ms ≈ 7.5 h	time
Full batch run	~26 h wall clock: no longer fits in a night	time
Article embedding matrix	193 MB grows with the corpus; cold starts get slower	cold-start latency

Table 8. Scale limits (SCALE_NOTES.md).

At 10x scale, the very first component to break is the exposure index due to a memory limit: storing ~2 billion impression records requires 16 GB of RAM, which exceeds our single machine's 11 GB capacity. The batch pipeline breaks on a time limit, as computing nightly profiles for ~8M users would take ~7.5 hours and full batch runs would take ~26 hours, exceeding the available overnight window. The online per-request scoring path scales much better because each query only inspects one user's data, although a growing article embedding matrix worsens cold-start latency. If we distribute across multiple machines, user profile data can easily be sharded by user ID since individual requests are independent, but the article corpus cannot be split by user and must be replicated on every machine so all candidates remain accessible.

5. Anti-gaming (Q9)

Dataset	Honest vs leaky (future features added)	Paired Δ
EB-NeRD	leaky scores higher	+0.0162 [0.0153, 0.0172]
MIND	honest 0.6144, leaky 0.5994	-0.0150

Table 9. What cheating with serving-unavailable features buys.

For Q9, I trained a deliberately cheating model using future-looking features (like total pageviews and future exposure windows) to measure how much leakage inflates performance. On EB-NeRD, cheating inflated the score by +0.0162 Δ AUC [0.0153, 0.0172], which illustrates how easily future information creates fake improvements offline. On MIND, the leaky model actually scored lower (0.5994 vs 0.6144), but this doesn't mean leakage is safe: the 'next 24 hours' feature got abruptly

truncated at the end of the test log, shifting feature values by 3.2x and confusing the tree splits. To guard against this, we built two automated unit tests: verifying that deleting all data after the prediction timestamp leaves features unchanged, and ensuring that shuffling click labels has zero effect on feature matrices, both of which caught all 4 planted test leaks.

6. What I learned

Working through this assignment showed me that offline numbers can easily be deceptive if you don't carefully audit how they were produced. Several times during experimentation, re-checking a result completely changed the conclusion: an apparent 665x speedup was partly caused by background machine load, and a 0.9222 AUC score turned out to be an artifact of candidate list exposure bias rather than actual ranking quality. Similarly, our 11M-parameter NRMS model looked +0.0198 better on validation but failed to generalize to the test split. Benchmarking serving latency also reinforced that meeting an SLA warm doesn't mean a system is production-ready if cold starts still blow past the limit, and real engineering requires testing across fresh restarts and honest splits.

AI usage

All implementation code was developed with AI assistance under my direct prompting, architectural direction, and debugging (documented in `AI_USAGE.md`).